

ЛЕКЦИЯ 12



ВЫЧИСЛИТЕЛЬНЫЕ СИСТЕМЫ. ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ ЧАСТЬ 1

- Уровни и средства комплексирования
- Классификация вычислительных систем
- Вычислительные системы класса MIMD



- **Вычислительная система (ВС)** – совокупность взаимосвязанных и взаимодействующих процессоров или вычислительных машин, периферийного оборудования и программного обеспечения
- Главное отличие ВС от ЭВМ – наличие нескольких вычислителей, реализующих *параллельную обработку*
- Цели создания ВС:
 - ☐ Повышение производительности
 - ☐ Повышение надёжности
 - ☐ Повышение достоверности вычислений
 - ☐ Предоставление дополнительных сервисных услуг



- По структуре ВС подразделяются на многомашинные и многопроцессорные
- **Многомашинная архитектура** состоит из множества классических ЭВМ, имеющих собственную локальную память, при этом отсутствует общая память ВС
 - ❑ Эффективны только при решении задач, разбиваемых на n слабо связанных подзадач (n – количество ЭВМ в ВС)
- **Многопроцессорная архитектура** позволяет организовать множество потоков данных и/или потоков команд
 - ❑ Параллельное выполнение нескольких фрагментов одной задачи



- **Микроуровень**. Выполнение команды разделяется на фазы, а фазы нескольких соседних команд могут быть перекрыты за счёт конвейеризации
- **Уровень команд**. Параллельно выполняется несколько команд (несколько конвейеров) – суперскалярные процессоры
- **Уровень потоков**. Задачи разбиваются на параллельно выполняемые потоки. Данный уровень достигается на параллельных ВС
- **Уровень заданий**. Несколько независимых заданий одновременно выполняются на разных процессорах, практически не взаимодействуя друг с другом (многопроцессорные и многомашинные ВС)



- **Гранулярность** – мера отношения объёма вычислений, выполненных в параллельной задаче, к объёму коммуникаций (для обмена сообщениями)
- Виды гранулярности:
 - ☐ Крупнозернистый параллелизм
 - ☐ Среднезернистый параллелизм
 - ☐ Мелкозернистый параллелизм
- Эффективность зависит от баланса между *степенью гранулярности* программ и *коммуникационной задержки*, возникающей между разными гранулами
- Если коммуникационная задержка минимальна, то выигрывает мелкоструктурное разбиение программы
- Если коммуникационная задержка велика, то выигрывает крупнозернистое разбиение программ



- Крупнозернистый параллелизм (coarse grained)
 - ❑ Распараллеливаются большие независимые программы (тысячи команд)
 - ❑ Обеспечивается операционной системой
- Среднезернистый параллелизм (medium grained)
 - ❑ Распараллеливаются независимые процедуры (сотни команд)
 - ❑ Обеспечивается программистом или компилятором
- Мелкозернистый параллелизм (fine grained)
 - ❑ Распараллеливаются группы команд (порядка 10)
 - ❑ Приблизительное равенство интенсивности вычислений и обмена данными
 - ❑ Обеспечивается распараллеливающим (векторизирующим) компилятором



Уровень параллелизма

Гранулярность

Микроуровневый параллелизм

Параллелизм уровня команд

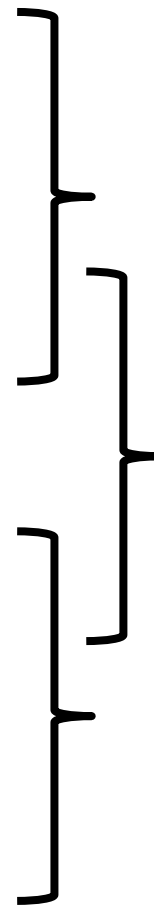
Параллелизм уровня потоков

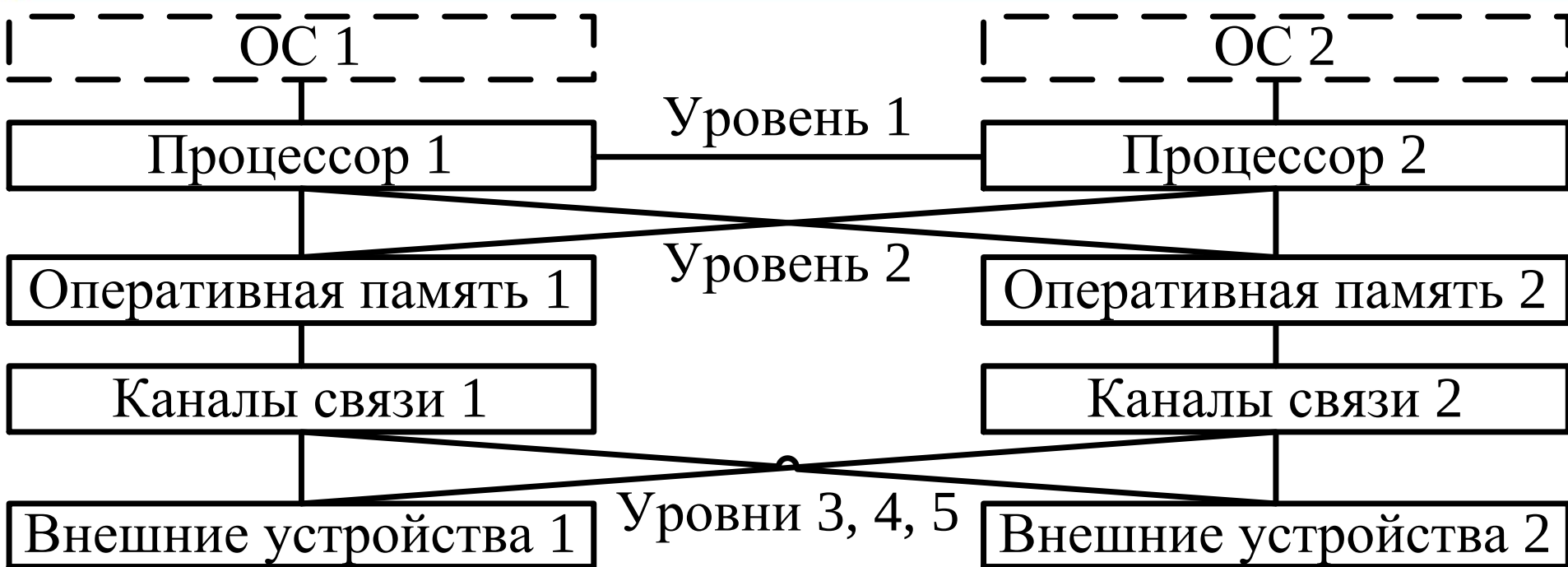
Параллелизм уровня заданий

Мелкозернистая

Среднезернистая

Крупнозернистая





1. Уровень прямого управления (процессор–процессор)
2. Уровень общей оперативной памяти
3. Уровень комплексируемых каналов ввода-вывода
4. Уровень устройств управления внешними устройствами
5. Уровень общих внешних устройств



➤ **Уровень прямого управления** служит для передачи коротких однобайтовых приказов-сообщений

1. Процессор-инициатор обмена передаёт в блок прямого управления байт-сообщение и подаёт команду «прямая запись»
2. У другого процессора эта команда вызывает внешнее прерывание
3. В ответ он вырабатывает команду «прямое чтение» и записывает передаваемый байт в свою память
4. После завершения передачи оба процессора продолжают вычисления по собственным программам



- **Уровень общей оперативной памяти** является наиболее предпочтительным для оперативного взаимодействия процессоров
- Обычно этот уровень дополняется первым уровнем, что позволяет повышать оперативность взаимодействия процессоров
- Центральным блоком такой системы является быстродействующий коммутатор, обеспечивающий необходимые подключения абонентов (процессоров и каналов) к общей оперативной памяти
- Уровень эффективно работает при небольшом числе обслуживаемых абонентов

- **Уровень комплексизируемых каналов ввода-вывода** служит для передачи больших объёмов данных между блоками оперативной памяти, сопрягаемых в ВС
- Обмен данными между ЭВМ осуществляется с помощью адаптера «канал–канал» и команд «чтение» и «запись».
Адаптер – это устройство, согласующее скорости работы сопрягаемых каналов
- Скорость обмена данными определяется скоростью самого медленного канала, что соответствует подключению периферийной аппаратуры через контроллеры и адаптеры



- **Уровень устройств управления внешними устройствами (УВУ)**
предполагает использование встроенного двухканального переключателя и команд «зарезервировать» и «освободить»
- Двухканальный переключатель позволяет подключать УВУ одной ВМ к селекторным каналам различных ВМ
- По команде «зарезервировать» канал-инициатор обмена имеет доступ через УВУ к любым ВЗУ. Обмен канала с ВЗУ продолжается до завершения работ и получения команды «освободить». Только после этого УВУ может подключиться к конкурирующему каналу
- Имеется возможность сопряжения с каналами связи – организация сетей ВМ



- Для подключения отдельных устройств используется автономный двухканальный переключатель
- Уровень общих внешних устройств используется в редких специальных случаях, когда в качестве внешнего объекта используется какое-то дорогое уникальное устройство. В противном случае этот уровень малоэффективен



- **Метрики параллельных вычислений** – система показателей, позволяющая оценить преимущества, получаемые при параллельном решении задачи на n процессорах, по сравнению с последовательным решением той же задачи на единственном процессоре
- n – количество процессоров, используемых для организации параллельных вычислений
- $O(n)$ – объём вычислений, выраженный через количество операций, выполняемых n процессорами в ходе решения задачи
- $T(n)$ – общее время вычислений (решения задачи) с использованием n процессоров

- **Степень параллелизма** (Degree of parallelism) – $P(t)$ – число процессоров, одновременно выполняющих программу в каждый момент времени

$$O(n) = \Delta \sum_{i=1}^n i t_i,$$

где t_i – интервал времени,
 в течение которого $P = i$,
 $\sum_{i=1}^n t_i = t_k - t_n$ – общее
 время вычислений



- **Индекс параллелизма** (*Parallel Index*) характеризует среднюю скорость параллельных вычислений через количество выполненных операций:

$$PI(n) = \frac{O(n)}{T(n)}$$

- **Ускорение** (*Speedup*) – отношение времени, требуемого для выполнения программы на одном процессоре ко времени параллельного вычисления на n процессорах

$$S(n) = \frac{T(1)}{T(n)}$$



- **Эффективность** (*Efficiency*) характеризует целесообразность наращивания числа процессоров через ту долю ускорения, достигнутого за счёт параллельных вычислений, которая приходится на один процессор:

$$E(n) = \frac{S(n)}{n} = \frac{T(1)}{nT(n)}$$

- **Утилизация** (*Utilization*) учитывает вклад каждого процессора при параллельном вычислении, но в виде количества операций, выполненных процессором в единицу времени

$$U(n) = \frac{O(n)}{nT(n)}$$



- **Избыточность** (*Redundancy*) – отношение объёма параллельных вычислений к объёму эквивалентных последовательных вычислений:

$$R(n) = \frac{O(n)}{O(1)}$$

- **Сжатие** (*Compression*) вычисляется как величина, обратная избыточности:

$$C(n) = \frac{O(1)}{O(n)}$$

- **Качество** (*Quality*):

$$Q(n) = \frac{T^3(1)}{nT^2(n)O(n)} = S(n)E(n)C(n)$$

$$n = 8; T(1) = O(1) = O(8) = 93; T(8) = 25$$

$$PI(8) = \frac{O(8)}{T(8)} = \frac{93}{25} = 3,72$$

$$S(8) = \frac{T(1)}{T(8)} = \frac{93}{25} = 3,72$$

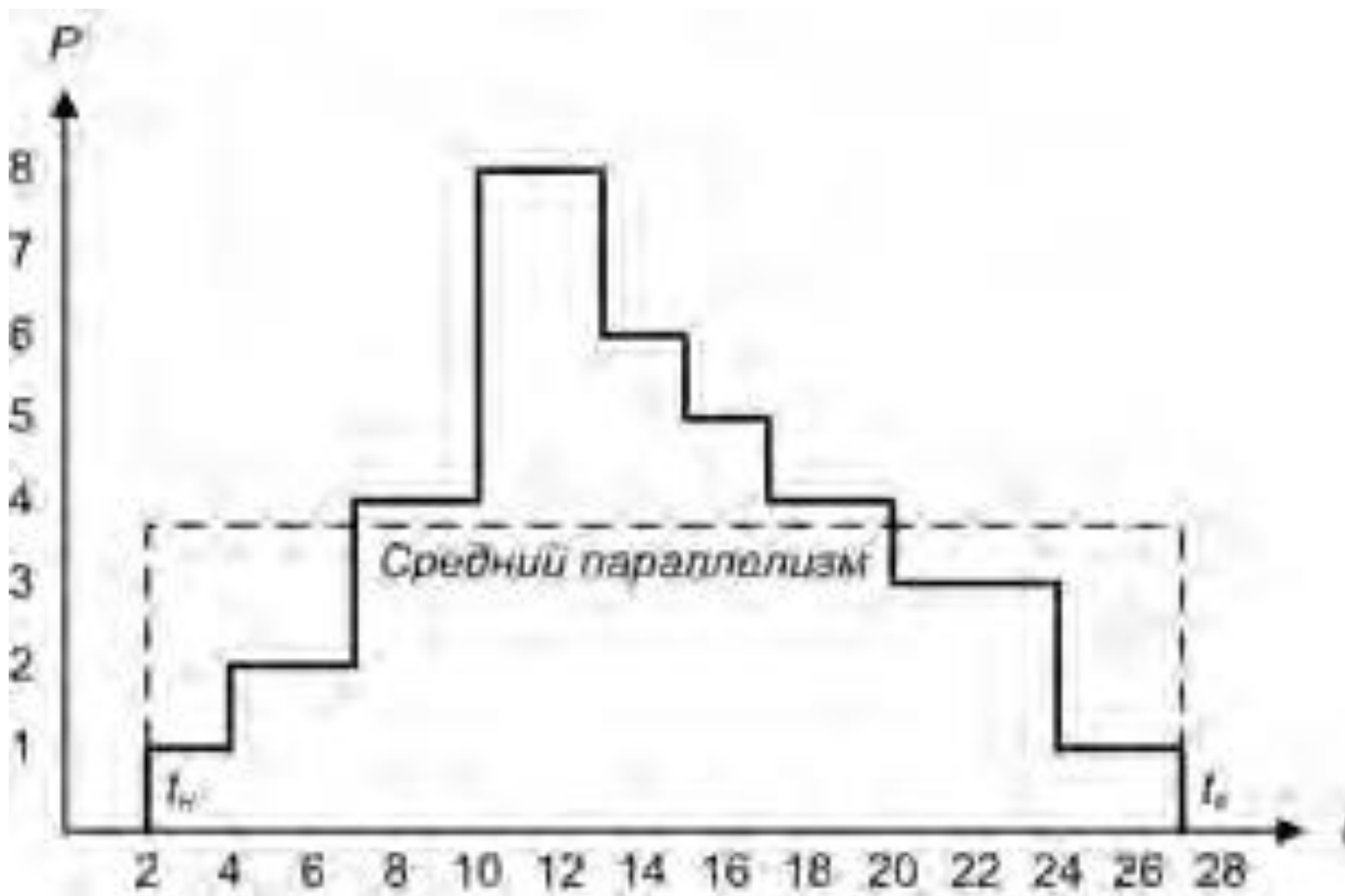
$$E(8) = \frac{T(1)}{8T(8)} = \frac{93}{8 \times 25} = 0,465$$

$$U(8) = \frac{O(8)}{8T(8)} = \frac{93}{8 \times 25} = 0,465$$

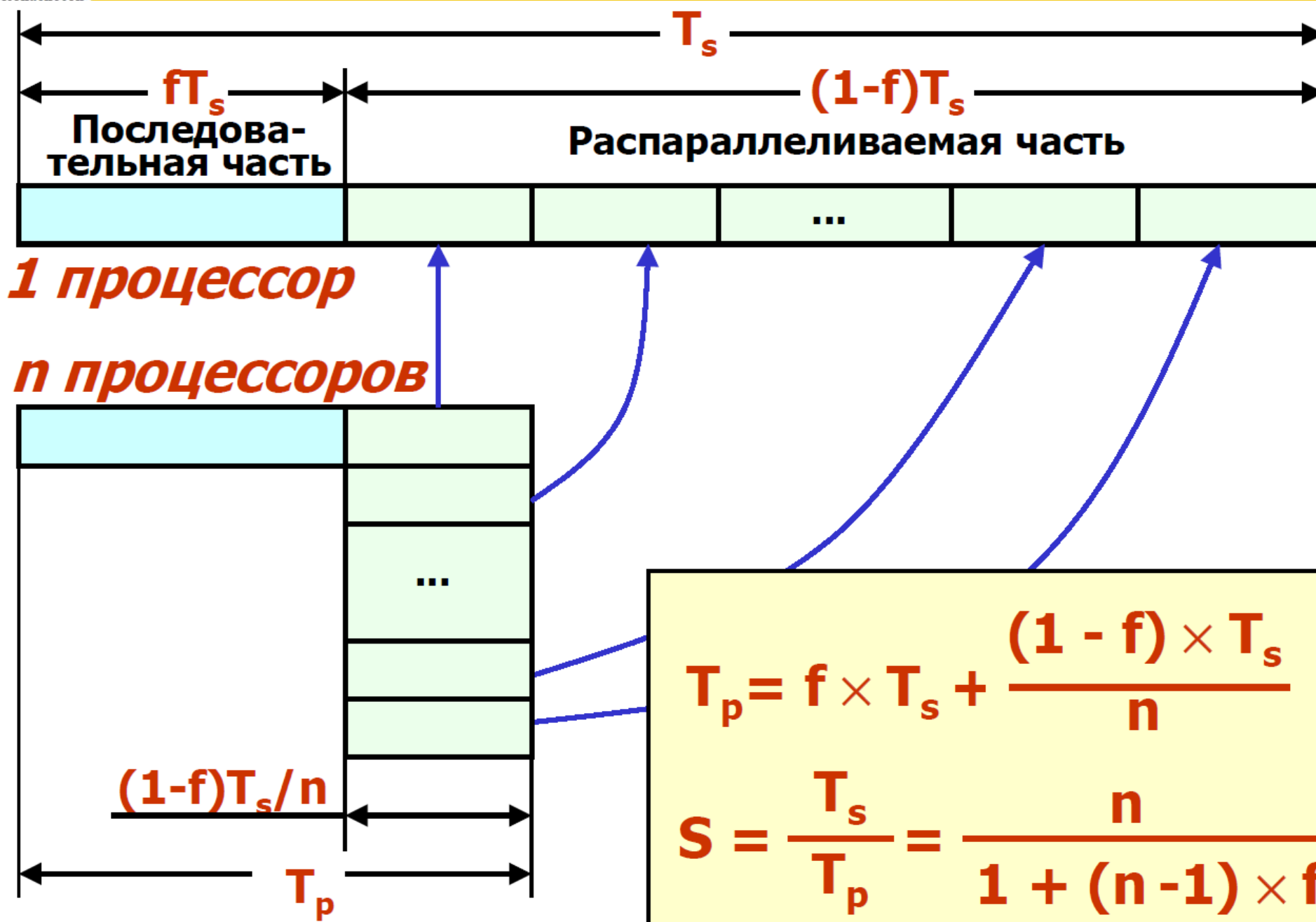
$$R(8) = \frac{O(8)}{O(1)} = \frac{93}{93} = 1$$

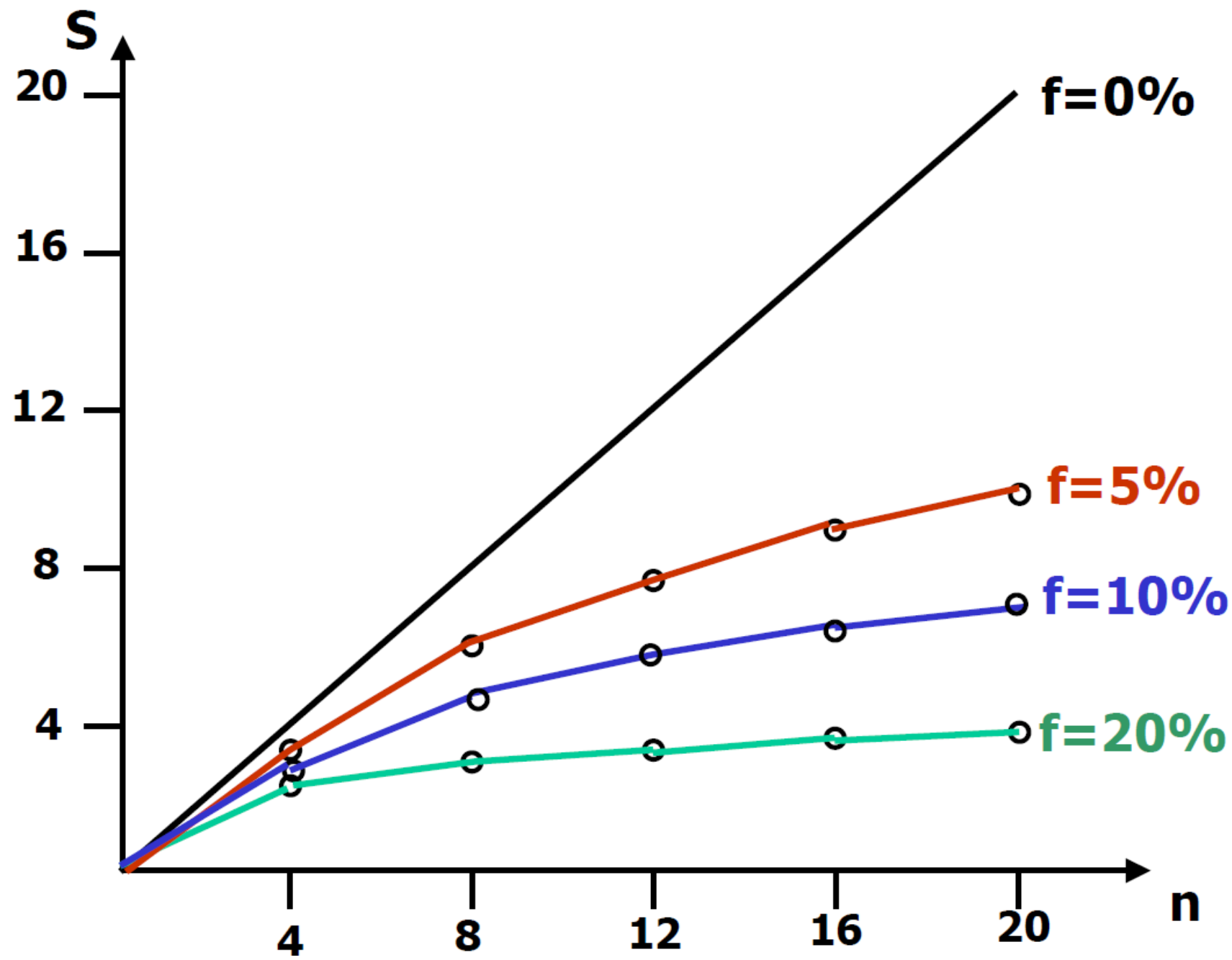
$$C(8) = \frac{O(1)}{O(8)} = \frac{93}{93} = 1$$

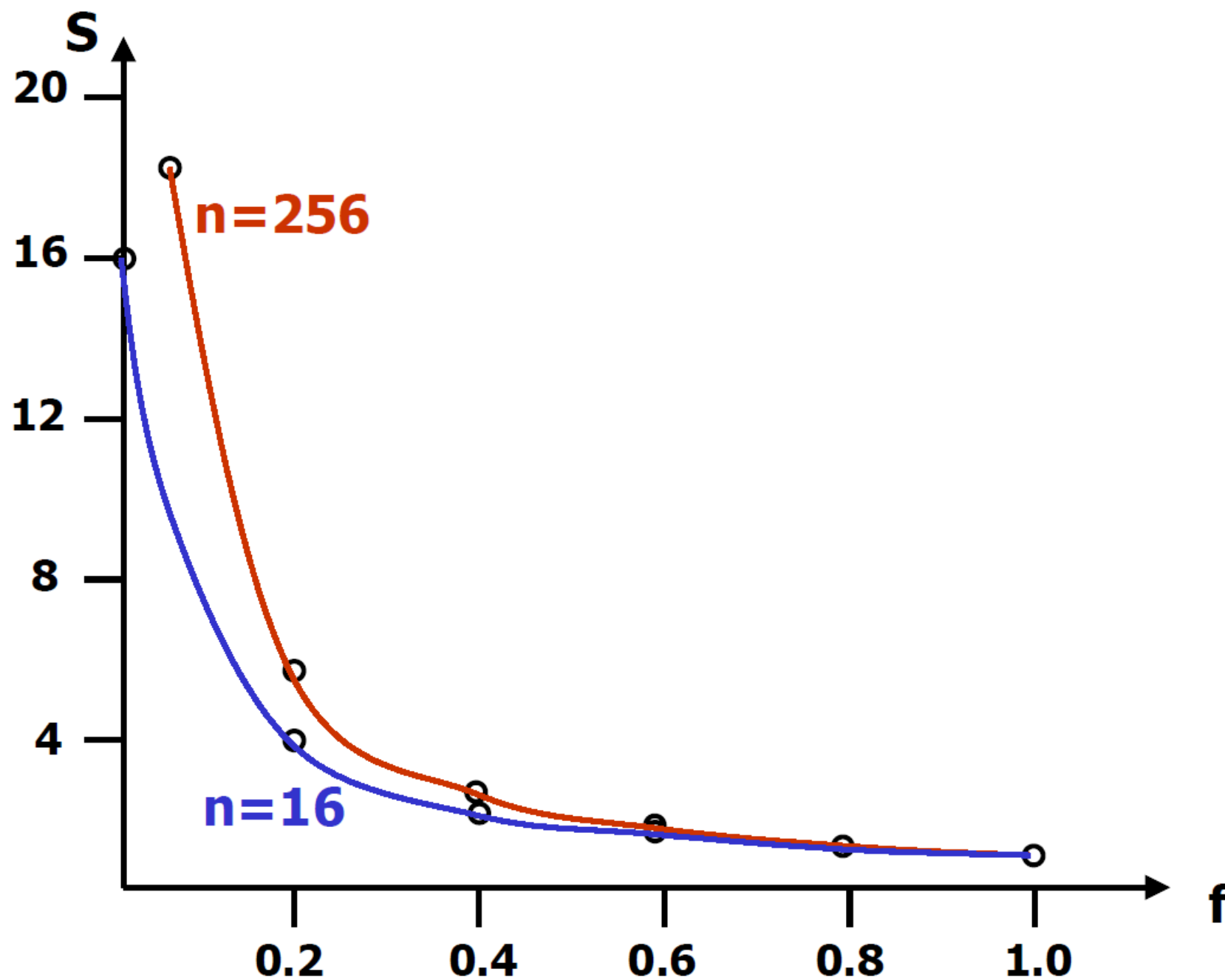
$$Q(8) = S(8)E(8)C(8) = 3,72 \times 0,465 \times 1 = 1,73$$



Закон Амдала





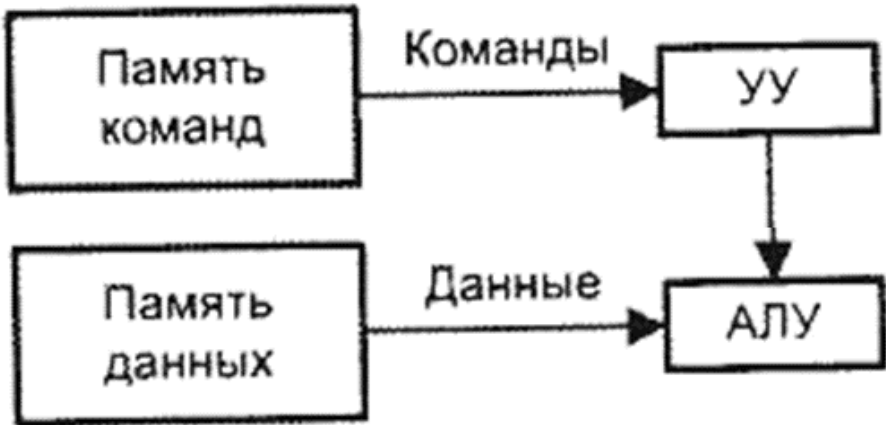
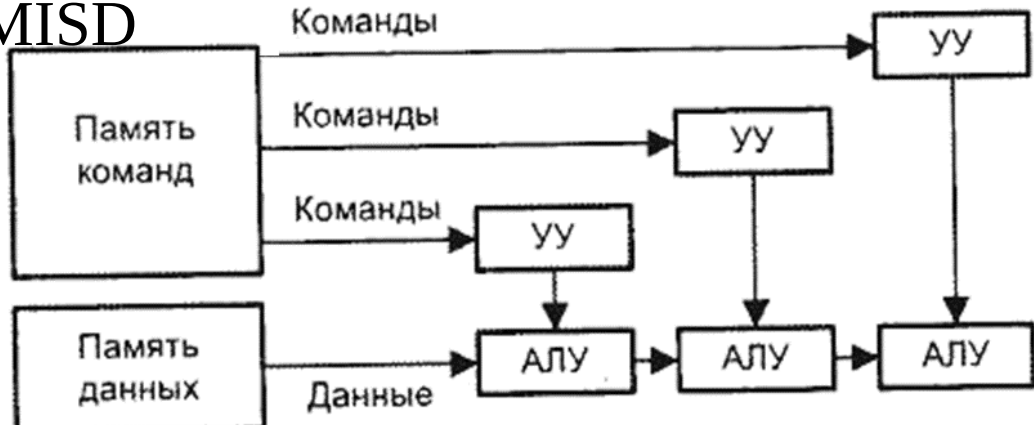
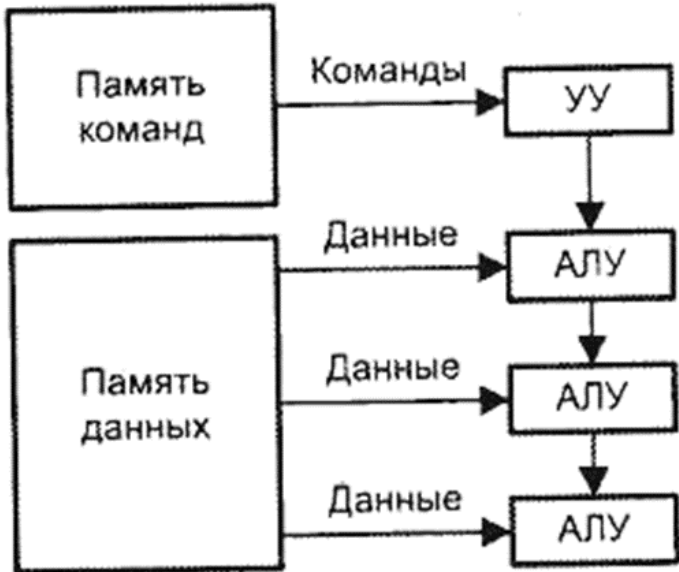
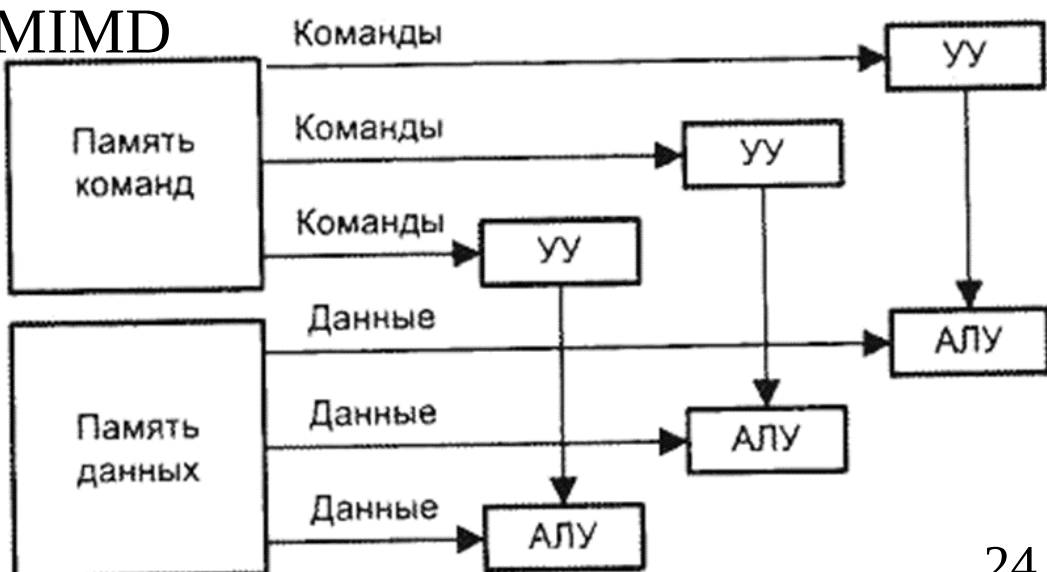




- Использование нескольких процессоров не приводит к пропорциональному приросту производительности
- Пример:
 - ❑ Одиночный процессор (1 x П) простаивает 20 % своего времени, ожидая данные из ОП
 - ❑ 2 x П будет простаивать 33 % времени
 - ❑ 4 x П – 50 %
 - ❑ В пересчёте на общую производительность:
 - 1 x П-система работает со скоростью 100 %
 - 2 x П-система – со скоростью 167 % (вместо 200 %)
 - 4 x П-система – со скоростью 250 % (вместо 400 %)



➤ Поток – это последовательность элементов, обрабатываемых процессором

Поток данных (Data Str.)	Поток команд (Instruction Stream)	
	Одиночный (Single)	Множественный (Multiple)
Одиночный (Single)	SISD 	MISD 
	SIMD 	MIMD 
Множественный (Multiple)		

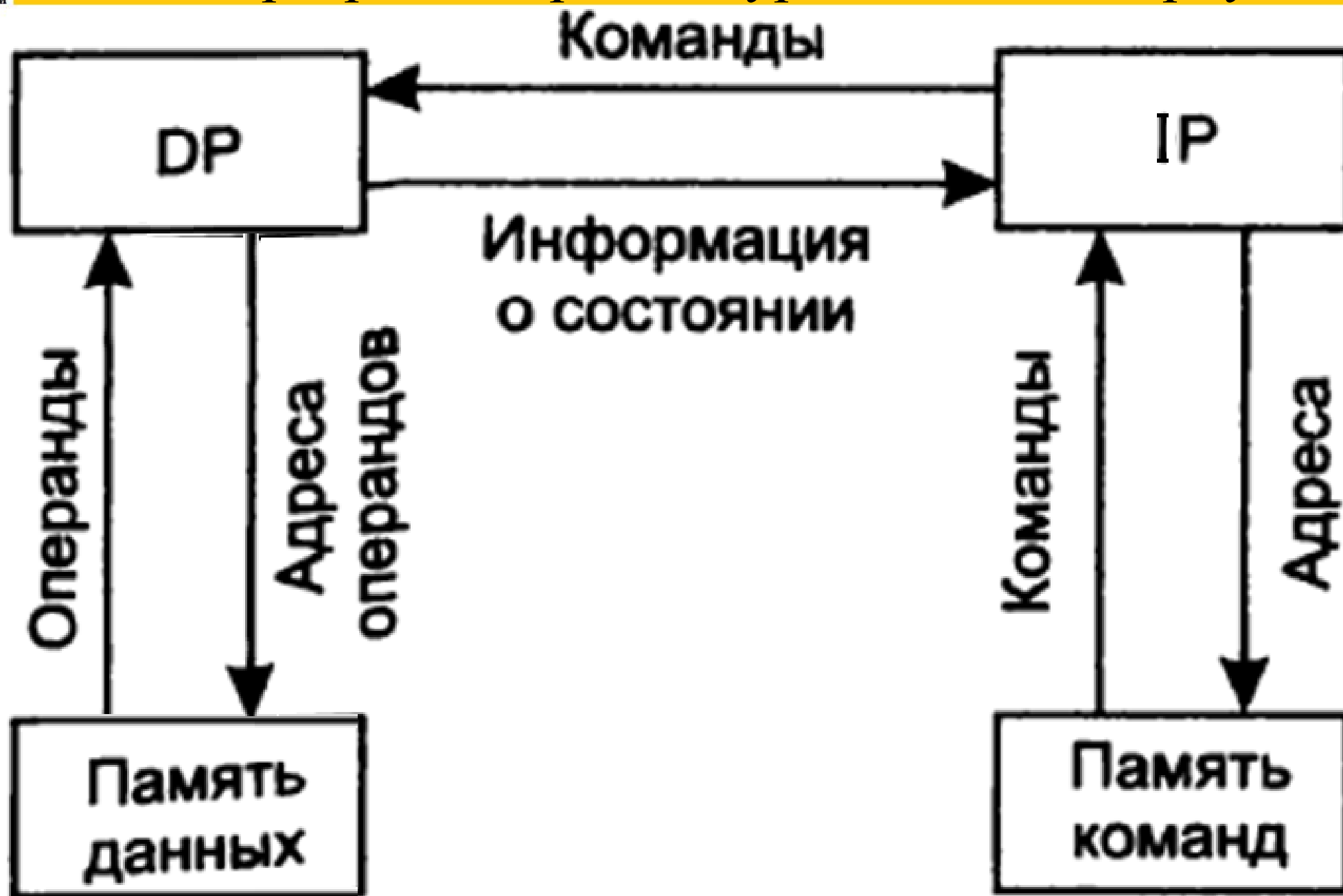
Обмен данными	Память	
	Общая	Распределённая
Общие данные	GMSV – General Memory-Shared variables (Общая память – разделяемые переменные) Класс 1. «Системы с разделяемой памятью»	DMSV – Distributed Memory, Shared variables (Распределенная память – разделяемые переменные) Класс 2. «Гибридная архитектура»
Передача данных	GMMP – General Memory, Message propagation (Общая память – передача сообщений)	DMMP – Distributed Memory, Message propagation (Распределённая память – передача сообщений) Класс 3. «Архитектуры с передачей сообщений»



- *Процессор команд* (IP – Instruction Processor) – функциональное устройство, работающее как интерпретатор команд (в системе, вообще говоря, может отсутствовать)
- *Процессор данных* (DP – Data Processor) – функциональное устройство, работающее как преобразователь данных, в соответствии с арифметическими операциями
- *Иерархия памяти* (IM – Instruction Memory, DM – Data Memory) – запоминающее устройство, в котором хранятся данные и команды, пересылаемые между процессорами
- *Переключатель* – абстрактное устройство, обеспечивающее связь между процессорами и памятью



- Функции *процессора команд* (IP – Instruction Processor):
 - ❑ На основе своего состояния и полученной от DP информации IP определяет адрес команды, которая будет выполняться следующей
 - ❑ Осуществляет доступ к IM для выборки команды
 - ❑ Получает и декодирует выбранную команду
 - ❑ Сообщает DP команду, которую надо выполнить
 - ❑ Определяет адреса операндов и посылает их в DP
 - ❑ Получает от DP информацию о результате выполнения команды
- Функции *процессора данных* (DP – Data Processor):
 - ❑ DP получает от IP команду, которую надо выполнить
 - ❑ Получает от IP адреса операндов
 - ❑ Выбирает операнды из DM
 - ❑ Выполняет команду
 - ❑ Запоминает результат в DM
 - ❑ Возвращает в IP информацию о состоянии после выполнения команды



- $1-1$ – связывает пару функциональных устройств
- $n-n$ – связывает i -е устройство из одного множества устройств с i -м устройством из другого множества (попарная связь)
- $1-n$ – переключатель соединяет одно выделенное устройство со всеми функциональными устройствами из некоторого набора
- $n \times n$ – каждое функциональное устройство одного множества может быть связано с любым устройством другого множества и наоборот



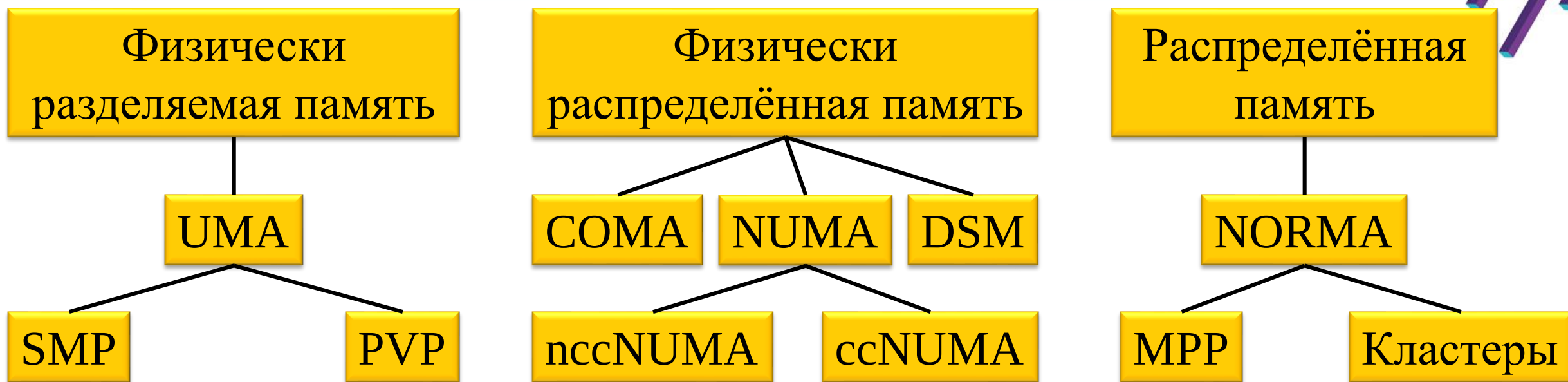
➤ Первый этап – 8 характеристик:

- ☐ Количество процессоров команд (IP)
- ☐ Число запоминающих устройств (модулей) команд (IM)
- ☐ Тип переключателя между IP и IM
- ☐ Количество процессоров данных (DP)
- ☐ Число запоминающих устройств (модулей) данных (DM)
- ☐ Тип переключателя между DP и DM
- ☐ Тип переключателя между IP и DP
- ☐ Тип переключателя между DP и DP

➤ Количественные характеристики имеют 3 возможных значения: {0; 1; n}

➤ Всего выделено 28 классов архитектур

- 1-5 – dataflow и reduction (*нет процессоров команд*)
- 6 – классическая гарвардская машина
- 7-10 – матричные процессоры
- 11-12 – MISD (не заполнены)
- 13-20 – привычные MIMD
- 21-28 – незаполненные MIMD



➤ **UMA** – Uniform Memory Access:

❑ **SMP** – Symmetric Multiprocessing

❑ **PVP** – Parallel Vector Process

➤ **NUMA** – Nonuniform Memory Access:

❑ **ccNUMA** – Correct Cashe NUMA

❑ **nccNUMA** – Non Correct Cashe NUMA

➤ **COMA** – Cache-Only Memory Architecture

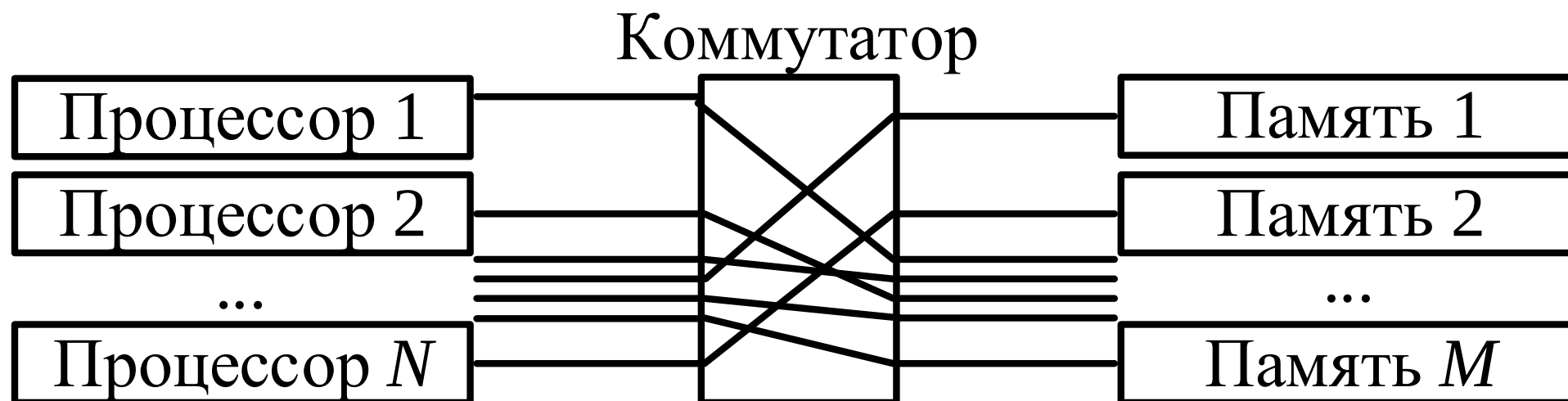
➤ **DSM** – Distribute Shared Memory

➤ **Norma** – No Remote Memory Access:

❑ **MPP** – Massive Parallel Processing

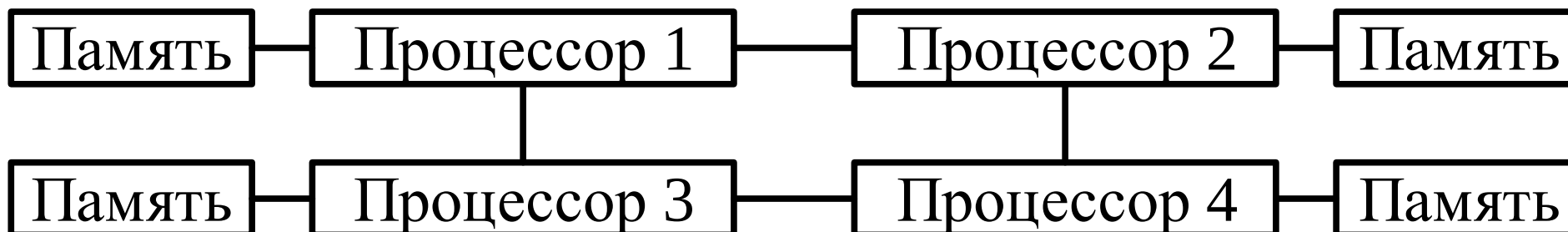
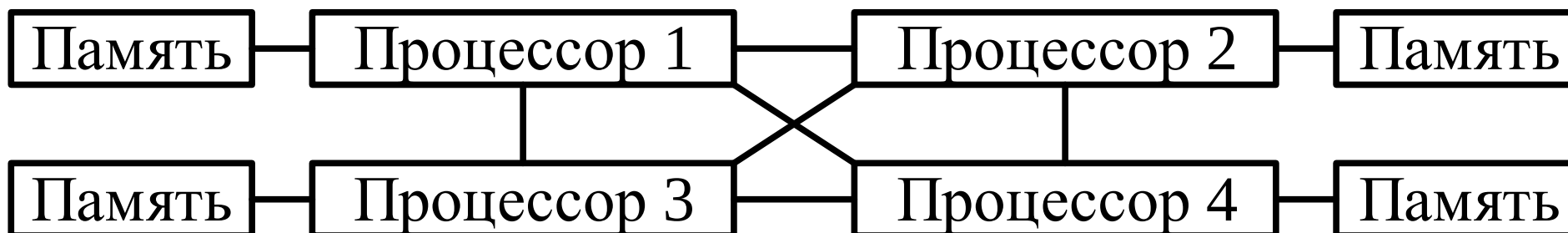
❑ Кластерная архитектура

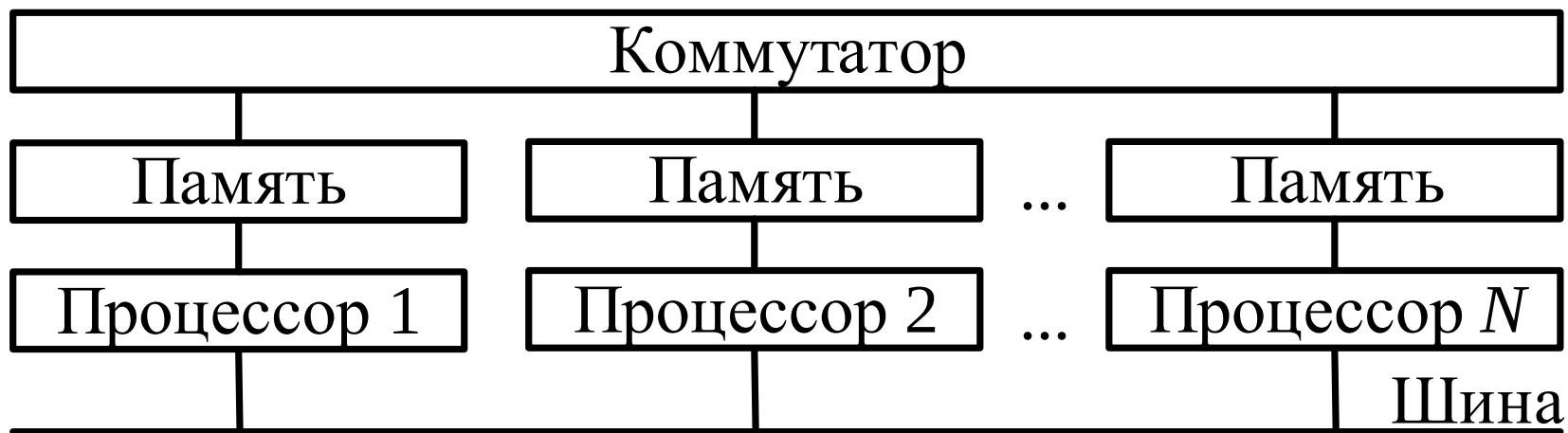
- Все процессоры имеют равные возможности по доступу к единому адресному пространству – доступ любого процессора к памяти производится единообразно и занимает одинаковое время
- UMA (Uniform Memory Access) – *системы с однородным доступом к памяти*
- Единая память может быть построена как одноблочная или по блочному принципу
- Технически UMA-системы предполагают наличие узла, соединяющего каждый из n процессоров с каждым из m банков памяти:
 - ❑ Объединение нескольких процессоров с банками разделяемой памяти посредством общей шины
 - ❑ Применение коммутатора, маршрутизирующего запросы процессора к одному из блоков памяти





- Каждый процессор имеет локальную память, а совокупность локальной памяти всех процессоров образует общую память ВС
- Единое адресное пространство и каждый из n процессоров имеет доступ к любой ячейке в пределах этого пространства
- Варианты реализации физически распределённой разделяемой памяти:
 - ☐ NUMA (Non-Uniform Memory Access)
 - ☐ COMA (Cache Only Memory Architecture)
 - ☐ DSM (Distribute Shared Memory)

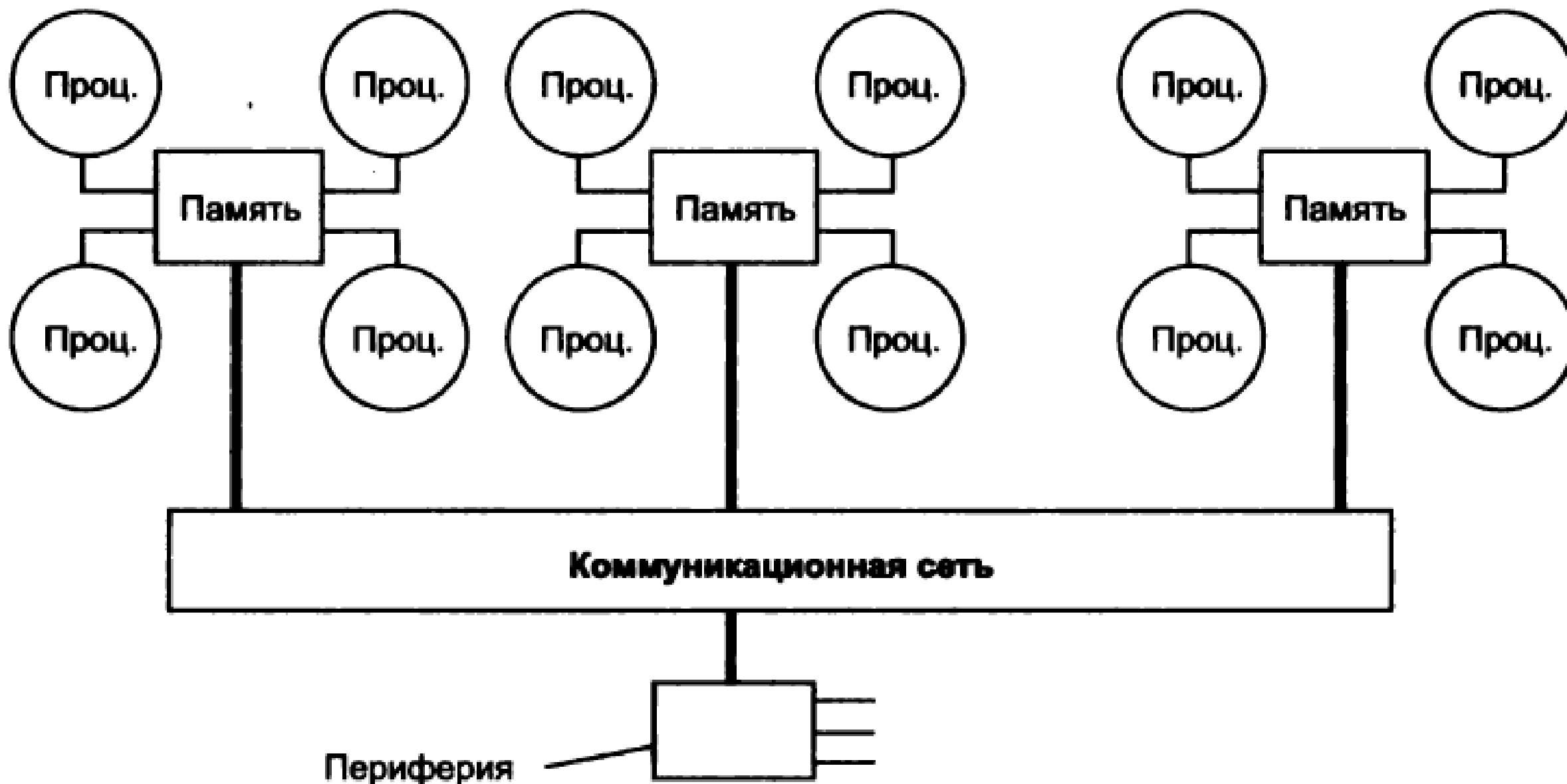


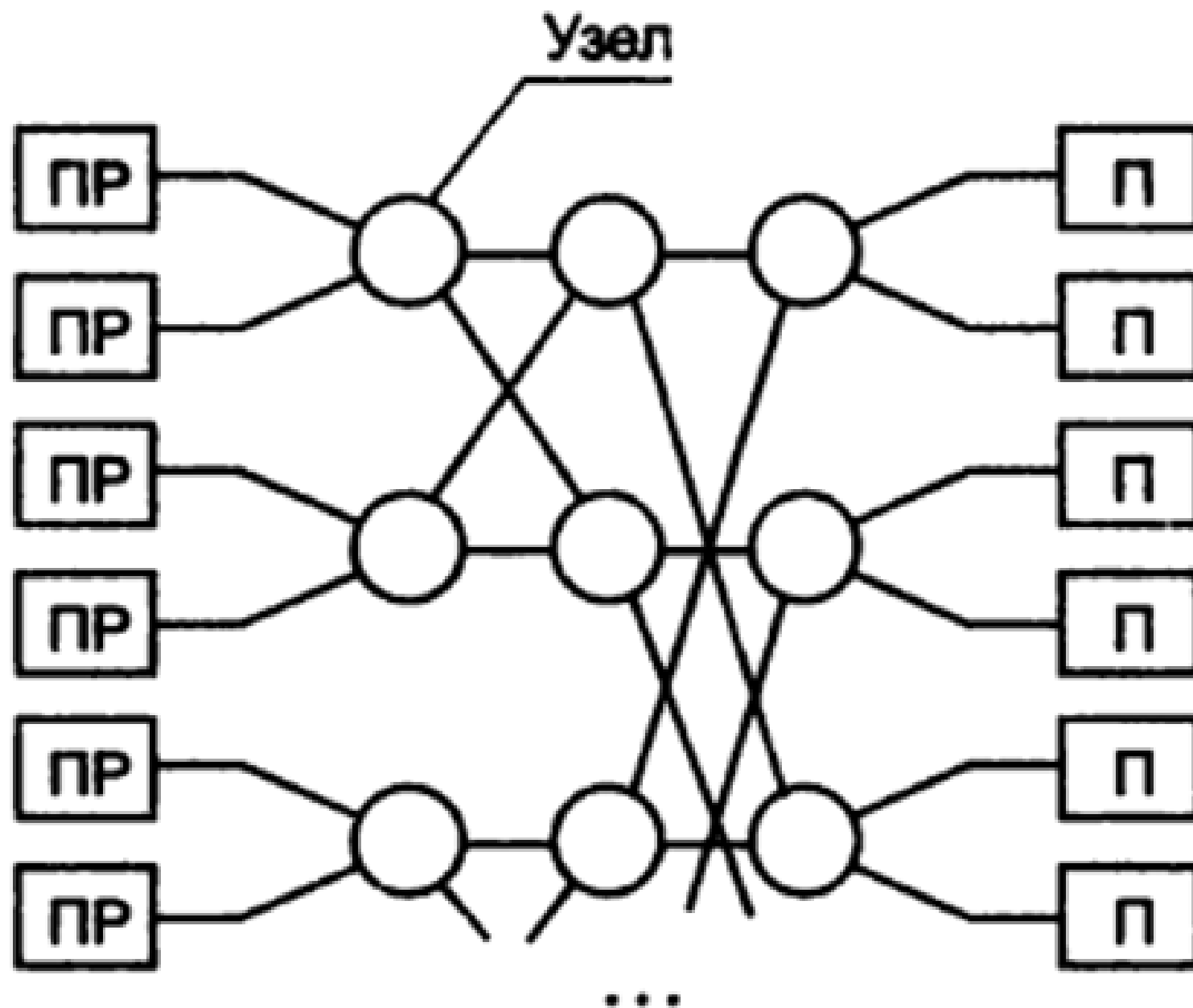


- NUMA (Non-Uniform Memory Access) – неоднородный доступ к памяти
- Локальной памяти процессоров выделен отдельный диапазон адресов в едином адресном пространстве
- Каждый процессор имеет возможность физически «добраться» до локальной памяти остальных процессоров (удалённой памяти)
- Доступ процессора к собственной локальной памяти производится напрямую (быстро), доступ к удалённой памяти через коммутатор или сеть (долго)



- Помимо локальной памяти каждый процессор имеет локальную кэш-память → проблема обеспечения непротиворечивости копий одного и того же блока ОП ВС во всех локальных кэшах (когерентности кэш-памяти)
- При изменении информации в кэш-памяти одного процессора копии аналогичного блока в кэш-памяти остальных процессоров должны быть откорректированы или помечены как недостоверные
- Подвиды архитектуры NUMA:
 - ❑ *ccNUMA (Cache Coherent Non-Uniform Memory Architecture) – неоднородный доступ к памяти с обеспечением когерентности кэшей*
 - ❑ *nccNUMA (Non-Cache Coherent Non-Uniform Memory Architecture) – неоднородный доступ к памяти без обеспечения когерентности кэшей*







- СОМА (Cache Only Memory Architecture) – архитектура только с кэш-памятью
- Совокупность кэш-памяти всех процессоров рассматривается как глобальная память ВС, причём классическая глобальная память отсутствует
- Данные переносятся в кэш-память того процессора, который последним их запросил, при этом расположение переменной в памяти не фиксировано уникальным адресом
- Перенос данных из одной локальной кэш-памяти в другую не требует участия ОС и обеспечивается аппаратным *справочником*, хранящим информацию о текущем расположении всех переменных
- Последняя копия элемента данных никогда не удаляется из кэш-памяти



- DSM (Distribute Shared Memory) – распределённая разделяемая память
- Каждый процессор не имеет возможности физического доступа к локальной памяти остальных процессоров
- ВС представляется пользователю как система с общей памятью лишь благодаря операционной системе
- Операционная система предлагает пользователю единое адресное пространство, однако фактическое обращение к памяти «чужого» компьютера обеспечивается путём обмена сообщениями между процессорами



- **Norma** (NO Remote Memory Access) – архитектуры без прямого доступа к удалённой памяти
- Доступ к удалённой памяти возможен только путём обмена сообщениями с процессором с адресуемой памятью
- Преимущества
 - ☐ не возникает конкуренция за шину или коммутаторы
 - ☐ размер системы ограничивает только сеть между процессорами
 - ☐ нет проблемы когерентности кэш-памяти
- Основной недостаток – сложность обмена информацией между процессорами. Два вида издержек:
 - ☐ На формирование и пересылку сообщения от одного процессора к другому
 - ☐ Для обеспечения реакции на сообщения от других процессоров принимающий процессор должен получить запрос прерывания и выполнить процедуру обработки этого прерывания



- По отношению к каждому процессору все остальные процессоры можно рассматривать просто как периферийные устройства
- Для посылки сообщения процессор формирует блок данных в своей локальной памяти и извещает свой локальный контроллер о необходимости передачи информации на другой процессор
- По коммуникационной сети это сообщение пересылается на приёмный контроллер принимающего процессора
- Последний находит место для сообщения в собственной локальной памяти и уведомляет процессор-источник о получении сообщения